software

MDPI

Article

Investigating Reproducibility Challenges in LLM Bugfixing on the HumanEvalFix Benchmark

Balázs Szalontai, Balázs Márton, Balázs Pintér * and Tibor Gregorics

Department of Software Technology, Faculty of Informatics, Eötvös Loránd University, Pázmány Péter sétány 1/C, H-1117 Budapest, Hungary; szalontaib@inf.elte.hu (B.S.); g68pwv@inf.elte.hu (B.M.); gt@inf.elte.hu (T.G.)
* Correspondence: pinter@inf.elte.hu

Abstract

Benchmark results for large language models often show inconsistencies across different studies. This paper investigates the challenges of reproducing these results in automatic bugfixing using LLMs, on the HumanEvalFix benchmark. To determine the cause of the differing results in the literature, we attempted to reproduce a subset of them by evaluating 12 models in the DeepSeekCoder, CodeGemma, CodeLlama, and WizardCoder model families, in different sizes and tunings. A total of 35 unique results were reported for these models across studies, of which we successfully reproduced 12. We identified several relevant factors that influenced the results. The base models can be confused with their instruction-tuned variants, making their results better than expected. Incorrect prompt templates or generation length can decrease benchmark performance, as well as using 4-bit quantization. Using sampling instead of greedy decoding can increase the variance, especially with higher temperature values. We found that precision and 8-bit quantization have less influence on benchmark results.

Keywords: reproducibility; bugfixing; humanevalfix; ml4code

check for updates

Academic Editor: Andrea Pinna

Received: 20 May 2025
Revised: 27 June 2025
Accepted: 7 July 2025
Published: 14 July 2025

Citation: Szalontai, B.; Márton, B.; Pintér, B.; Gregorics, T. Investigating Reproducibility Challenges in LLM Bugfixing on the HumanEvalFix Benchmark. *Software* **2025**, *4*, 17. https://doi.org/10.3390/software4030017

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Large language models (LLMs) have demonstrated remarkable capabilities in software development [1–3], which include their ability to detect and resolve bugs. Various benchmarks are designed to evaluate these capabilities, such as QuixBugs [4], HumanEvalFix [1], BugsInPy [5], and MDEVAL [6].

The most widely acknowledged bugfix benchmark is probably HumanEvalFix, a component of the HumanEvalPack [1]. In this benchmark, each of the 164 canonical solutions from the original HumanEval code generation benchmark [7] has been manually corrupted by introducing a bug. The task of the LLM is to repair the buggy function. The modified output is then verified using the original test cases.

Although reliable reports of benchmark results are essential to compare models, it is not uncommon to see differing benchmark results for the same model throughout studies [8–12]. In the broader community, researchers can struggle to reproduce the officially reported benchmark results. The difference between the reported and reproduced results can be minor, but it can also be significant. This not only complicates the comparison of models but also raises questions about the validity of the currently published benchmark results.

In this paper, we address the issue of inconsistent benchmark results on the HumanEvalFix benchmark. We look for discrepancies in a broad range of results published

in the literature, considering models with scores reported in two or more studies. To identify the underlying reasons for these discrepancies, we conduct our own evaluations using 12 models of various sizes and tunings: DeepSeekCoder (1.3B and 6.7B, base and instruct) [3], CodeLlama (7B and 13B, base and instruct) [2], CodeGemma (2B base, 7B base and instruct) [13], and WizardCoder (15B) [14]. Our findings are as follows:

- We reproduce 16 of the 32 scores reported for the evaluated models. This implies that many of the discrepancies in the reported results can be attributed to using different or possibly incorrect evaluation settings.
- We quantify the impact of modifying the evaluation settings individually. For instance, the maximum generation length, sampling strategy, 4-bit quantization, and prompt template choice significantly influence the results, whereas precision and 8-bit quantization do not.
- We identify instances of possibly misreported results, likely due to confusion between base and instruction-tuned models.

2. Related Work

Some studies have raised questions about the reproducibility and reliability of benchmark results of LLMs, identifying sources of variance that can undermine them. A clear consensus in the field is that even minor differences in evaluation methods and settings might lead to significant discrepancies.

Biderman et al. investigated the impact of different prompts on model performance, demonstrating their significant influence on benchmark results [9]. To address the reproducibility issues, they propose a checklist for reporting benchmark results. This checklist suggests disclosing the evaluation setup and sharing the exact prompts, the evaluation code, and the model outputs. Furthermore, they recommend against copying results from other papers without verification. They propose the Language Model Evaluation Harness framework as a tool to facilitate greater reproducibility.

Laskar et al. point to the lack of transparent evaluation details [10]. They highlight the issue of not disclosing the precise model versions, which can lead to inconsistencies. The authors also address potential problems originating from the prompt design, including the practice of "prompt hacking", manipulating input prompts to elicit desired responses. They also emphasize the lack of transparency concerning decoding parameters, such as the temperature, and note issues related to excessive limitations on generation length.

Hochlehnert et al. studied the heightened sensitivity of reasoning models to decoding parameters [11]. They propose a standardized framework for model evaluation with clearly defined best practices and reporting standards. Applying this framework revealed decreased performance for several models, suggesting that their previously reported results might have been influenced by optimized evaluation parameters. Their work also investigated the impact of random seeds, showing that single-seed evaluations can be unstable. They averaged results over multiple seeds to obtain a more accurate representation of performance. They observed that higher temperature values can increase the accuracy but also lead to greater variability, while higher top_p values improve the performance without compromising the stability. Other factors contributing to performance variance are also discussed, such as differences in hardware settings and software frameworks.

Yuan et al. emphasized the crucial role of numerical precision in reproducible LLM evaluation [12]. They recommend using fp32 precision to reduce rounding errors. They also measured the extent to which factors like the GPU count, GPU hardware version, and evaluation batch size influence benchmark results. They find that bf16 precision is sensitive to variations in hardware and system configurations. To mitigate this, they propose an optimized inference pipeline called LayerCast, which stores model weights in

16-bit precision but performs all computations in fp32, thereby balancing memory efficiency with numerical stability.

3. Method

We begin by introducing the benchmark we focus on, HumanEvalFix, as well as the evaluation framework commonly used to run it. Then, we identify the discrepancies across the reported results by conducting a literature review of the benchmark scores. Finally, we analyze GitHub issues on the unsuccessful reproduction of published results to determine possible reasons for these discrepancies and our experimental setup.

3.1. Benchmarking with HumanEvalFix

The authors of the HumanEvalFix benchmark have introduced a bug into each of the 164 canonical solutions in the HumanEval benchmark, thereby turning it into a bug-fixing benchmark. These bugs involve missing logic, extra logic, or incorrect logic. The benchmarked LLM is prompted with the incorrect function, along with unit tests, and is tasked with repairing the function. The output code is then validated using the same unit tests found in the original HumanEval benchmark. Since the Python-based variant of this benchmark is the most widely used, we also concentrate our efforts on it.

For conducting our evaluations, we use the *Code Generation LM Evaluation Harness* [15] framework, provided by the *BigCode Project*. This framework includes multiple benchmark implementations, such as HumanEval, MBPP, and QuixBugs, as well as the HumanEvalFix benchmark. Most of these benchmarks evaluate the functional correctness of the code outputs using the pass@k metric, which indicates the percentage of outputs that pass the tests at least once in k attempts. Researchers often use a larger number of samples (>k) to provide a more refined pass@k estimate. Pass@1 is often reported, which measures whether the first generated output passes all test cases. Users of the framework can adjust a range of evaluation settings, such as the maximum generation length, batch size, and sampling parameters. The maximum generation length controls how many tokens the model can produce in a single output (including its prompt tokens). The batch size determines how many inputs are processed simultaneously. The precision settings (*fp16*, *bf16* and *fp32*) determine how numbers are represented during computation. Quantization reduces the model size by compressing the numerical representations, often with minimal performance loss. Sampling parameters include the temperature, which affects the randomness in output. Top-p restricts generation to the most probable tokens whose cumulative probability meets a threshold, helping control diversity.

3.2. Review of Reported Benchmark Results

We conducted a review of 17 papers that contained results of the HumanEvalFix benchmark [1,16–30], focusing on their reported HumanEvalFix results. These papers reported benchmark results for a total of 117 different models (including variants in sizes and tunings). Out of the 117 models mentioned, 25 appeared in at least two papers. These models formed the basis of our investigation. Their reported results are visualized in Figure 1, as well as in Table 1, with all relevant citations.

Six out of 25 models have identical reported results across studies (e.g., Instruct-CodeT5+, StarCoder). This suggests that the initially published result was cited by other authors without trying to reproduce it. For 4/25 models, the results differed only slightly (e.g., OctoCoder, Llama3-Inst-8B). Small differences are accepted in many cases as they can be due to minor differences in configuration settings. However for most of the models, more precisely in 15/25 cases, the results differed significantly (e.g., CodeLlama, DeepSeek-Coder). For example, the reported results for DeepSeekCoder-Instruct (1.3B) ranged from

9.1% to 48.9%. Similarly, for CodeLlama-Instruct (13B), six benchmark results are reported, ranging from 15.2% to 30.5%, nearly evenly spread. According to one reported result, DeepSeekCoder-Instruct (33B) should be one of the top models for bugfixing with an exceptionally high score of 81.0%; however, two other studies report a much lower score of 47.5%.

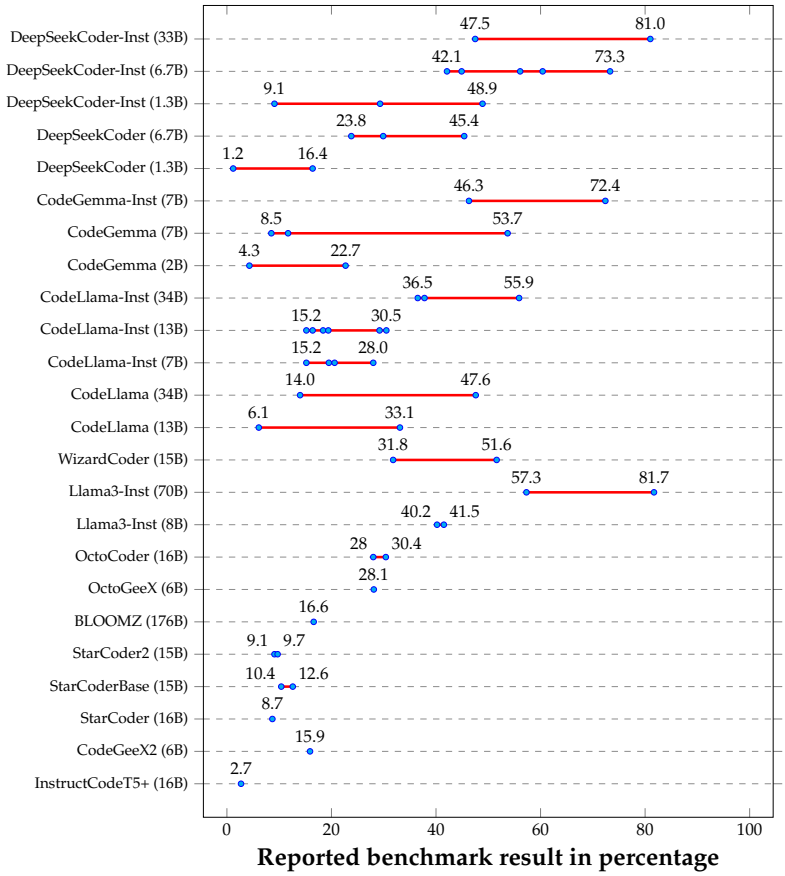

Figure 1. Reported HumanEvalFix results of LLMs. Only models with reported results from at least two papers are included. The labeled blue dots represent the results. Red lines are used in between the results to visualize the discrepancy between the minimum and maximum reported scores. In the original papers, multiple prompts were used for StarCoderBase and StarCoder2; however, we include only the results from the instruct prompt, as this is the one used by other authors.

Table 1. All reported HumanEvalFix results of LLMs considered in this paper. Citations are provided for studies introducing models and for ones that evaluate them.

Model	Reported Results
DeepSeekCoder-Inst (33B) [3]	47.5 [16,22], 81.0 [27]
DeepSeekCoder-Inst (6.7B) [3]	42.1 [21], 44.9 [16,22], 56.1 [30], 60.4 [17], 73.3 [27]
DeepSeekCoder-Inst (1.3B) [3]	9.1 [16], 29.3 [21], 48.9 [27]
DeepSeekCoder (6.7B) [3]	23.8 [21], 29.9 [30], 45.4 [27]
DeepSeekCoder (1.3B) [3]	1.2 [21], 16.4 [27]
CodeGemma-Inst (7B) [13]	46.3 [23], 72.4 [27]
CodeGemma (7B) [13]	8.5 [23], 11.7 [27], 53.7 [17]
CodeGemma (2B) [13]	4.3 [23], 22.7 [27]
CodeLlama-Inst (34B) [2]	36.5 [16,22], 37.8 [23], 55.9 [27]

Table 1. *Cont.*

Model	Reported Results
CodeLlama-Inst (13B) [2]	15.2 [20], 16.4 [24], 18.9 [23], 19.4 [22], 29.2 [30], 30.5 [27]
CodeLlama-Inst (7B) [2]	15.2 [24], 19.5 [23], 20.6 [27], 28.0 [30]
CodeLlama (34B) [2]	14.0 [23], 47.6 [27]
CodeLlama (13B) [2]	6.1 [23], 33.1 [27]
WizardCoder (15B) [14]	31.8 [1,20,24,30], 51.6 [27]
Llama3-Inst (70B) [31]	57.3 [23], 81.7 [27]
Llama3-Inst (8B) [31]	40.2 [23], 41.5 [27]
OctoCoder (16B) [1]	28 [23], 30.4 [1,20,22,24,25,30]
OctoGeeX (6B) [1]	28.1 [1,24]
BLOOMZ (176B) [32]	16.6 [1,20,25]
StarCoder2 (15B) [22]	9.1 [23], 9.7 [22,25]
StarCoderBase (15B) [33]	10.4 [23], 12.6 [22,25]
StarCoder (16B) [33]	8.7 [1,20,24,30]
CodeGeeX2 (6B) [34]	15.9 [1,20,24]
InstructCodeT5+ (16B) [35]	2.7 [1,20,24]

A factor that complicates cross-study comparisons is that models are sometimes referenced with slightly different parameter counts. For example, two papers list a specific version of WizardCoder as a 15B model [14,30], another as 15.5B [20], two others as 16B [1,24], and another as both 15B and 15.5B [27]. Although these differences probably originate from varied rounding/flooring, they nonetheless hinder model identification. In our work, we treat this version of WizardCoder as a 15B parameter model.

3.3. Experimental Setup

The goal of our experiments was twofold. First, we examined the effect of a wide range of settings on the results of models mentioned in multiple studies. Then, we tried to reproduce the results reported in the literature by determining the evaluation settings they could have used. In this section, we describe the experimental setup for the evaluations.

To explore potential causes for the differences in the benchmark results, we reviewed the reported issues about unsuccessful attempts at reproducing published results in the GitHub repository of the benchmarking framework (https://github.com/bigcode-project/bigcode-evaluation-harness). At the time of investigation, 145 issues were published (47 open and 98 closed). We reviewed these issues on the 5th of November 2024. Thirteen out of the 145 issues (six open and seven closed) were about the unsuccessful reproduction of officially reported results. We analyzed the issues that were either closed or had helpful answers to them (10 in total), and found the following causes (links to these issues are provided in the Appendix A.2):

- A differently tuned version of the model was used (e.g., the base model instead of its instruction-tuned variant), causing a large difference in benchmark result. (Two issues)
- The wrong variant of the same benchmark was used: MBPP instead of MBPP+ and HumanEval instead of its unstripped variant. (Two issues)
- The temperature was set incorrectly: while generating one sample for each prompt, the temperature $T = 0.8$ turned out to be an improperly large value. (One issue)

- An incorrect prompt was used, causing more than 6% of a decrease in pass@1 performance. (One issue)
- The reproduced results were different by only a few percentage points compared to the officially reported ones. Such a discrepancy was considered negligible, as this can be due to minor variations in evaluation settings, minor updates in model versions, or in hardware configurations. (Three issues)

Most papers provide details about their evaluation setups, but these can sometimes be insufficient for full reproducibility. For instance, the precision used during evaluation is never specified in the papers. Quantization is also generally unmentioned, although this may simply indicate that it was not used and thus is less problematic. The authors of the HumanEvalFix benchmark [1] used a decoding setup with temperature $T = 0.2$, $top_p = 0.95$, and generated 20 samples to estimate pass@1 performance. Some other authors adopt these settings in their setups [16,20,22,27]. Some papers report using greedy decoding, without further details [21,23,25]. A few papers explicitly mention using the *Code Generation LM Evaluation Harness* [21,22,25,27], the framework released by the benchmark's original team. As for the prompts used for evaluation on HumanEvalFix, most authors report their settings [1,16,21–23,25,27]. However, some papers have major shortcomings in reporting any of the evaluation settings described above for this benchmark [17,24,30].

For our local evaluations, we selected models that appear in at least two papers and also exhibited notable differences in benchmark results. Models larger than 15B parameters were excluded due to hardware limitations. While our main focus is on instruction-tuned models—as these are more frequently evaluated throughout studies—, we also take the results of the base models into consideration to assess whether they reproduce results originally attributed to their instruction-tuned counterparts. Consequently, although CodeLlama-7B does not meet the inclusion criteria, we do include it in our evaluations because it serves as the base model for CodeLlama-7B-Instruct. The selected models are as follows: DeepSeekCoder (1.3B and 6.7B), DeepSeekCoder-Instruct (1.3B and 6.7B), CodeLlama (7B and 13B), CodeLlama-Instruct (7B and 13B), CodeGemma (2B and 7B), CodeGemma-Instruct (7B), and WizardCoder (15B). We evaluated these models on the HumanEvalFix benchmark, using the following evaluation settings and values:

Prompt template: The evaluation framework defaults to the *instruct* prompt template, with only the context and instruction (the program and the instruction to fix it). The default prompt template lacks the model-specific formatting suggested by model authors. We evaluate models both with their suggested prompt template as well as the default *instruct* setting. All prompt settings used in our study are detailed in Appendix A.1.

Sampling vs. greedy decoding: The default behavior in the framework is not to use greedy decoding, but to apply sampling with the temperature $T = 0.2$. Alongside greedy decoding, we conduct experiments using sampling with temperatures $T = 0.2$ and $T = 0.8$.

Limiting generation length: The default value of 512 tokens is insufficient, as it can lead to unfinished programs and reduced benchmark scores. We conducted our experiments using lengths of 512 and 2048. Since the *max length generation* parameter considers both the prompt and the generated output, it must be large enough to prevent cutting the output short. We found 2048 to be a good choice on this benchmark, as tokenizers typically fit the inputs within 1024 tokens, leaving enough space for the generated output.

Precision and quantization: The majority of LLMs are released using bf16 precision, making it the preferred choice for precision. We evaluated the models using all three common precision formats: fp16, fp32, and bf16. While quantization is a useful tech-

nique to lower memory usage when running models, it is generally best to use models without it. In addition to our tests without quantization, we also ran experiments using 4-bit and 8-bit quantization (with fp16 precision).

Other settings: Further settings (such as *seed*), were kept at their default values as defined by the framework for all evaluations. Specifically, the default value for *seed* is 0.

To assess the effect of each parameter setting individually, we selected a configuration as the baseline. Then, we compared the evaluation results of this baseline configuration with those obtained when only one setting differed. In the baseline setting, the prompt format followed the recommendations of the model's authors, no sampling was applied, the maximum generation length was set to the sufficiently large value of 2048, and bf16 precision was used without quantization.

Next, we attempted to reproduce the results reported in the literature. To achieve this, we ran an exhaustive grid search for each model, with all possible combinations of the evaluation settings. As we had two options for prompt, three for sampling and temperature, two for generation length, and five for precision and quantization, we evaluated every single model in 60 settings. Although some reports originated from generating $n = 20$ samples with $T = 0.2$ to provide a more refined estimation of pass@1 performance, it would be too time-consuming for our grid search approach.

The experiments were performed on A100 GPUs with 40GB VRAM each. We set *max_memory_per_gpu* to the value of *auto*, which distributes the model on the assigned GPUs. We utilized only as many GPUs as required to load the model and execute the benchmark, which is typically just one. Where multiple GPUs were used, we also measured whether modifying the number of assigned GPUs had an influence on results; we did not see any variation in benchmark results when increasing the number of utilized GPUs.

4. Experimental Results

In this section, we present the results of our evaluations. First, we examined the impact of each evaluation parameter individually, compared to the baseline configuration. Next, we performed an exhaustive grid search for each model and assessed whether the reported results could be reproduced. All evaluation results are summarized in Figure 2.

4.1. The Effect of Individual Evaluation Settings

We measured the effect of individual evaluation settings by comparing the baseline configuration against variants in which evaluation parameters are modified individually. The effect of each parameter change on the benchmark scores was analyzed in isolation. The impact of these modifications is visualized in Figure 3.

4.1.1. Temperature: Greedy Evaluation and Sampling

Considering all models, sampling with temperature $T = 0.2$ has a slight impact on the average performance: -0.82%. The change in performance is more significant when using $T = 0.8$: $+5.48\%$. However, by only considering the instruction-tuned models, this change is less notable: $+0.34\%$ (with $T = 0.2$) and $+1.34\%$ (with $T = 0.8$).

For the baseline of greedy decoding, the standard deviation of scores across precision and quantization settings is 1.49 for all models and 1.74 for instruction-tuned models. When we switch to sampling at $T = 0.2$, the standard deviation rises to 1.86 ($+25.19\%$) for all models and 2.23 ($+28.70\%$) for the instruction-tuned ones. Increasing the temperature further to $T = 0.8$ increases the standard deviation even more, to 2.36 ($+58.54\%$) for all models and 2.89 ($+66.64\%$) for the instruction-tuned models.

Sampling + Temperature:				Greedy decoding				Sampling with T = 0.2				Sampling with T = 0.8				Reproduction of reports
Max generation length:				2048		512		2048		512		2048		512		match / no match
Prompt type:				proper	instruct	proper	instruct	proper	instruct	proper	instruct	proper	instruct	proper	instruct	close match (±1%)
Model			Precision													(close) match in other tuning
DeepSeekCoder	Instruct	1.3B	fp16	21.34%	20.73%	**16.46%**	18.90%	22.56%	20.12%	**16.46%**	18.90%	**29.27%**	25.00%	17.68%	18.29%	9.1%
			fp32	21.34%	20.73%	**16.46%**	18.90%	21.34%	20.12%	**16.46%**	18.90%	25.00%	25.61%	17.68%	23.17%	29.3%
			bf16	20.73%	22.56%	15.85%	20.73%	20.73%	23.78%	17.68%	19.51%	25.61%	24.39%	17.68%	21.95%	48.9%
			fp16 (8bit)	20.73%	19.51%	FAILED	FAILED	20.12%	24.39%	FAILED	FAILED	20.73%	28.05%	FAILED	FAILED	
			fp16 (4bit)	21.95%	20.12%	FAILED	FAILED	22.56%	20.73%	FAILED	FAILED	20.12%	23.78%	FAILED	FAILED	
		6.7B	fp16	48.78%	**45.12%**	31.71%	37.20%	51.22%	**45.12%**	32.93%	37.20%	49.39%	43.29%	31.71%	33.54%	42.1%
			fp32	48.78%	**45.12%**	31.71%	37.20%	48.78%	**44.51%**	32.93%	37.80%	50.00%	**42.68%**	34.76%	35.37%	44.9%
			bf16	50.00%	46.34%	32.93%	37.20%	50.61%	**45.73%**	32.32%	37.80%	50.61%	**44.51%**	34.76%	34.76%	56.1%
			fp16 (8bit)	50.61%	**44.51%**	FAILED	FAILED	52.44%	**44.51%**	FAILED	FAILED	52.44%	40.24%	FAILED	FAILED	60.4%
			fp16 (4bit)	**44.51%**	39.02%	FAILED	FAILED	43.29%	39.02%	FAILED	FAILED	**44.51%**	39.63%	FAILED	FAILED	73.3%
	Base	1.3B	fp16	0.61%	**1.22%**	0.00%	0.61%	0.61%	2.44%	0.00%	1.83%	3.66%	8.54%	1.83%	6.71%	1.2%
			fp32	0.61%	**1.22%**	0.00%	0.61%	0.61%	2.44%	0.00%	1.83%	4.27%	10.98%	1.83%	8.54%	16.4%
			bf16	0.61%	**1.22%**	0.00%	0.61%	0.61%	1.83%	0.00%	1.83%	3.66%	11.59%	1.83%	7.93%	
			fp16 (8bit)	0.61%	3.66%	FAILED	FAILED	0.61%	4.27%	FAILED	FAILED	2.44%	4.88%	FAILED	FAILED	
			fp16 (4bit)	1.83%	6.10%	FAILED	FAILED	3.05%	6.10%	FAILED	FAILED	3.05%	10.98%	FAILED	FAILED	
		6.7B	fp16	30.49%	19.51%	25.61%	17.07%	31.10%	20.12%	25.61%	18.29%	32.32%	27.44%	**24.39%**	25.00%	23.8%
			fp32	31.10%	19.51%	26.22%	17.07%	31.10%	20.12%	25.61%	18.29%	32.93%	27.44%	26.83%	19.51%	29.9%
			bf16	31.71%	20.73%	25.61%	17.68%	**29.88%**	21.34%	**24.39%**	19.51%	34.15%	**24.39%**	25.00%	25.61%	45.4%
			fp16 (8bit)	30.49%	20.12%	FAILED	FAILED	33.54%	20.73%	FAILED	FAILED	**24.39%**	25.61%	FAILED	FAILED	
			fp16 (4bit)	22.56%	17.07%	FAILED	FAILED	21.34%	21.34%	FAILED	FAILED	35.37%	**23.17%**	FAILED	FAILED	
CodeLlama	Instruct	7B	fp16	**19.51%**	24.39%	17.68%	22.56%	18.29%	23.78%	15.85%	21.95%	**19.51%**	21.95%	17.07%	25.00%	15.2%
			fp32	**19.51%**	24.39%	17.68%	22.56%	18.29%	23.78%	16.46%	21.95%	21.34%	21.95%	16.46%	23.78%	19.5%
			bf16	**19.51%**	24.39%	17.07%	22.56%	18.90%	22.56%	15.85%	23.17%	17.07%	22.56%	15.85%	**20.73%**	20.6%
			fp16 (8bit)	21.34%	25.61%	FAILED	FAILED	21.95%	24.39%	FAILED	FAILED	24.39%	24.39%	FAILED	FAILED	28.0%
			fp16 (4bit)	**15.24%**	**15.24%**	FAILED	FAILED	**15.24%**	**15.24%**	FAILED	FAILED	14.02%	21.34%	FAILED	FAILED	
		13B	fp16	**18.90%**	12.80%	17.68%	12.20%	18.29%	12.20%	**16.46%**	13.41%	**19.51%**	21.34%	20.12%	13.41%	15.2% / 30.5%
			fp32	**18.90%**	12.80%	17.68%	12.20%	18.29%	12.20%	17.07%	13.41%	**19.51%**	23.17%	18.29%	17.07%	16.4%
			bf16	**18.90%**	12.80%	17.68%	12.20%	**19.51%**	14.02%	17.68%	13.41%	21.95%	**19.51%**	20.73%	**15.24%**	18.9%
			fp16 (8bit)	**18.90%**	13.41%	FAILED	FAILED	20.12%	15.85%	FAILED	FAILED	21.95%	20.73%	FAILED	FAILED	19.4%
			fp16 (4bit)	12.20%	10.37%	FAILED	FAILED	12.80%	11.59%	FAILED	FAILED	14.63%	17.07%	FAILED	FAILED	29.2%
	Base	7B	fp16	0.61%	15.85%	0.61%	14.63%	0.61%	14.63%	0.61%	14.02%	2.44%	17.68%	1.83%	15.85%	30.0%
			fp32	0.61%	15.85%	0.61%	14.63%	0.61%	14.63%	0.61%	14.02%	1.83%	15.85%	1.83%	19.51%	
			bf16	0.61%	15.24%	0.61%	14.02%	0.61%	14.02%	0.61%	14.63%	0.00%	15.24%	1.83%	16.46%	
			fp16 (8bit)	0.61%	14.63%	FAILED	FAILED	0.61%	15.85%	FAILED	FAILED	3.05%	21.34%	FAILED	FAILED	
			fp16 (4bit)	0.61%	14.63%	FAILED	FAILED	0.61%	13.41%	FAILED	FAILED	1.22%	18.90%	FAILED	FAILED	
		13B	fp16	0.61%	4.27%	0.61%	14.63%	0.61%	3.05%	0.61%	3.66%	3.66%	12.20%	3.05%	13.41%	6.1%
			fp32	0.61%	3.66%	0.61%	14.63%	0.61%	3.05%	0.61%	3.66%	4.88%	12.20%	3.05%	13.41%	33.1%
			bf16	0.61%	5.49%	0.61%	4.88%	0.61%	**6.10%**	0.61%	5.49%	3.66%	9.15%	2.44%	17.07%	
			fp16 (8bit)	0.61%	7.32%	FAILED	FAILED	0.61%	4.88%	FAILED	FAILED	4.88%	15.24%	FAILED	FAILED	
			fp16 (4bit)	0.61%	7.32%	FAILED	FAILED	0.00%	6.71%	FAILED	FAILED	3.66%	15.24%	FAILED	FAILED	
CodeGemma	Instruct	7B	fp16	46.95%	34.15%	38.41%	29.27%	45.73%	33.54%	39.02%	29.27%	45.12%	32.93%	34.15%	26.83%	46.3%
			fp32	45.12%	33.54%	37.20%	28.66%	**46.34%**	33.54%	36.59%	28.05%	44.51%	35.98%	37.80%	28.05%	72.4%
			bf16	42.07%	34.15%	35.37%	28.66%	42.07%	29.88%	38.41%	27.44%	41.46%	33.54%	31.71%	26.83%	
			fp16 (8bit)	45.12%	32.93%	FAILED	FAILED	48.78%	32.93%	FAILED	FAILED	44.51%	35.98%	FAILED	FAILED	
			fp16 (4bit)	44.51%	29.27%	FAILED	FAILED	45.12%	32.32%	FAILED	FAILED	43.29%	30.49%	FAILED	FAILED	
	Base	2B	fp16	0.61%	**4.27%**	0.61%	**4.27%**	0.00%	4.88%	0.00%	4.88%	3.66%	7.32%	2.44%	6.71%	4.3%
			fp32	0.61%	4.88%	0.61%	4.88%	0.00%	4.88%	0.00%	3.66%	3.66%	10.37%	3.05%	6.10%	22.7%
			bf16	1.83%	**4.27%**	1.83%	**4.27%**	1.83%	**4.27%**	1.22%	4.88%	3.66%	9.15%	**4.27%**	12.20%	
			fp16 (8bit)	0.61%	4.88%	FAILED	FAILED	0.00%	4.88%	FAILED	FAILED	1.22%	9.15%	FAILED	FAILED	
			fp16 (4bit)	2.44%	4.88%	FAILED	FAILED	2.44%	4.88%	FAILED	FAILED	3.66%	9.76%	FAILED	FAILED	
		7B	fp16	0.61%	7.32%	0.61%	6.71%	1.22%	9.76%	0.61%	**8.54%**	7.93%	14.02%	6.10%	**12.20%**	8.5%
			fp32	0.61%	**8.54%**	0.61%	7.93%	1.22%	9.76%	0.61%	9.15%	4.88%	17.07%	5.49%	15.24%	11.7%
			bf16	5.49%	9.76%	4.88%	**8.54%**	4.88%	10.98%	3.05%	10.98%	5.49%	15.24%	6.10%	12.80%	53.7%
			fp16 (8bit)	0.61%	7.32%	FAILED	FAILED	1.22%	10.37%	FAILED	FAILED	6.71%	12.80%	FAILED	FAILED	
			fp16 (4bit)	0.00%	1.22%	FAILED	FAILED	0.61%	2.44%	FAILED	FAILED	0.00%	1.83%	FAILED	FAILED	
Wizard Coder	Instruct	15B	fp16	29.88%	29.88%	29.88%	29.88%	**31.71%**	**32.32%**	26.22%	25.61%	**31.71%**	28.05%	28.05%	28.05%	31.8%
			fp32	29.88%	29.88%	26.22%	27.44%	**31.71%**	28.05%	26.22%	25.61%	**32.32%**	30.49%	28.05%	28.05%	51.6%
			bf16	30.49%	29.27%	26.83%	27.44%	30.49%	29.88%	28.66%	27.44%	27.44%	32.93%	26.22%	30.49%	
			fp16 (8bit)	30.49%	26.83%	FAILED	FAILED	29.27%	28.66%	FAILED	FAILED	29.88%	29.88%	FAILED	FAILED	
			fp16 (4bit)	31.10%	29.27%	FAILED	FAILED	30.49%	28.66%	FAILED	FAILED	32.93%	28.66%	FAILED	FAILED	

Figure 2. All of our evaluation results, as well as the reported results for the evaluated models. Successful or near-exact reproductions of reported scores are highlighted with bold and underlined font. Greener cell background color indicates better performance. On the right side of the table, the reported results are colored to reflect their reproducibility: *cyan* if we were able to reproduce the score, *blue-green* if our results are within ±1%, *purple* if our score is within ±1% of a different model tuning than the one originally reported, and *red* if the score could not be reproduced. These colors are also described in the top-right corner of the table.

4.1.2. Maximum Generation Length

In the case of DeepSeekCoder-Instruct (1.3B) (evaluated using fp16), we see a notable decrease in performance (21.34% → 16.46%) when limiting the length of generation to the default value (to 512). This is a significant effect with a relative drop of 22.87%. The effect also applies more generally: across all models, the average performance drop is 19.45%. Similarly, for the instruction-tuned models, this drop is 19.80%.

4.1.3. Using an Incorrect Prompt Template

Considering the instruction-tuned models, switching from the suggested template to the default *instruct* prompt template resulted in average relative drop of 6.71% in model performance. This phenomenon is not observable when base models are also included in the calculation: the average performance even increases by 1.64%.

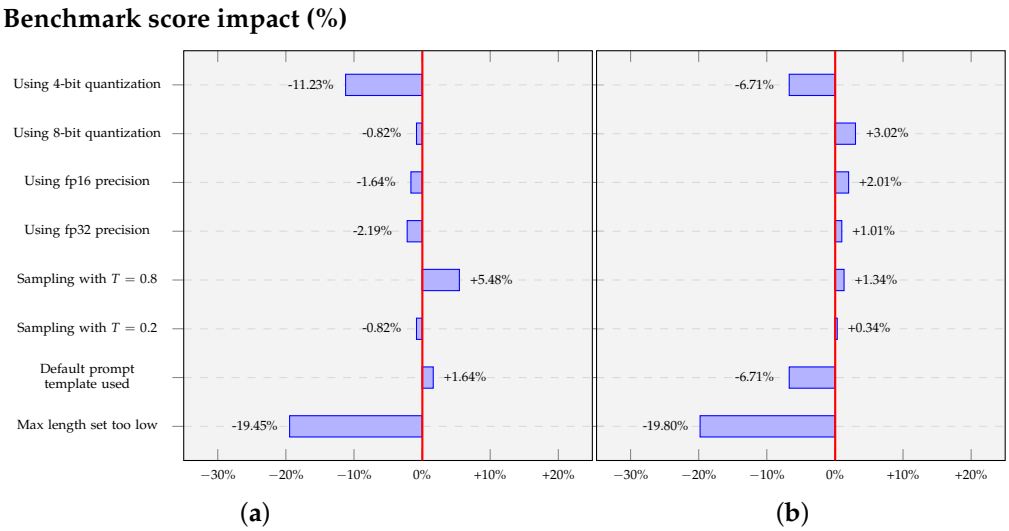

Figure 3. Effects of individually altering each evaluation setting relative to the baseline, showing the average change in the overall benchmark score. (**a**) All evaluated models; (**b**) instruction-tuned models.

4.1.4. Precision and Quantization

By switching from bf16 to fp32 or fp16 precision, the performance drops by 2.19% (fp32) and 1.64% (fp16) for all models and increases by 1.01% (fp32) and 2.01% (fp16) for instruction-tuned models. 8-bit quantization seems to have a similarly small effect with changes of −0.82% (all models) and +3.02% (instruction-tuned). However, using 4-bit quantization decreases the overall scores by a significant margin: −11.23% for all models and −6.71% for the instruction-tuned ones.

4.2. Reproducibility of Results in Existing Research

We attempted to reproduce existing results in the literature by trying out all possible combinations of the experimental settings. Our main goal was to identify potential misconfigurations in the original setups. All results from our evaluations as well as the scores from the existing studies can be seen in Figure 2.

4.2.1. CodeLlama

The results reported in studies for CodeLlama-Instruct (7B) range from 15.2% to 28.0%. This is in line with our evaluations, in which this model achieves scores from 14.02% to 25.61%. We reproduced all of the reported scores with one exception, indicating that the differences in reported results are indeed caused by variations in the evaluation settings. For CodeLlama-Instruct (13B), the reported results range from 15.2% to 30.5%. Our evaluation yielded considerably lower scores: 10.37% to 23.17%. We reproduced four out of six results.

The base CodeLlama (13B) model has two reported scores across papers: 6.1%, which we reproduced, and an unexpectedly high score of 33.1%. The latter score is very close to the upper range of scores reported for CodeLlama-Instruct (13B).

4.2.2. DeepSeekCoder

The reported results for the DeepSeekCoder (1.3B and 6.7B) base models are as high as 16.4% and 45.4% for the 1.3B and 6.7B variants, respectively. These scores are unusually high, especially when considering our own evaluations, which never exceed 11.59% (1.3B) and 35.37% (6.7B).

DeepSeekCoder (1.3B), for example, is measured to have both a 1.2% and 16.4% pass@1 score, which represents more than a 10× difference. We evaluated this model and its instruction-tuned variant and reproduced both reported results: 1.2% was reproduced

using the base model, while the score of 16.4% was only reproducible with the instruction-tuned variant. Using the 6.7B base model, we reproduced two of the reported results, but not the very high score of 45.4%. Its instruction-tuned variant however, has yielded multiple results close to this value, with the closest being 45.12%.

For the instruction-tuned 1.3B model, one of three values was reproduced. The score of 9.1% might come from the base model, as some of our evaluations yielded values close to it. Our evaluation results of the 6.7B instruction-tuned model range from 31.71% and 52.44%. These are partially in line with the reported results, with two very close results.

4.2.3. CodeGemma

Our evaluations of the CodeGemma (2B) base model attained up to 12.20% for the 2B model with one of the two reported scores reproduced. Using the 7B base model, values reach 17.07% with one exact match and one near match to the reported scores; the third score was significantly higher. In our evaluations of CodeGemma-Instruct (7B), we reproduced one of the two reported results.

4.2.4. WizardCoder

Our local evaluations of WizardCoder (15B) were close to one of the reported values (31.8%), with five near matches. However, our evaluations did not achieve even close to the other reported value of 51.6%.

5. Discussion

The results of our reproducibility experiments highlight several possible reasons for the discrepancies between the results in the literature. They also point to some strategies that could help prevent these discrepancies.

In some cases, we see surprisingly large scores for base models. We think this is due to confusing the base model with its instruction-tuned variant. It is crucial to ensure that the intended model variant is used, not one with different tuning. Furthermore, it is necessary to confirm that the correct model name and type is specified when reporting the evaluation results.

The pass@1 scores do not change significantly when switching from greedy evaluation to sampling, especially for instruction-tuned models. However, the standard deviation increases, especially with higher temperature values. Thus, we observe that greedy evaluation should be preferred over sampling when calculating the pass@1 performance on this task.

Restricting the models to a short generation length affects the results negatively, resulting in large drops in performance. In order to obtain accurate results, a proper limit should be chosen to avoid cutting possibly correct generations. In the case of our benchmark of focus, HumanEvalFix, 2048 is such a value, but this might differ for other benchmarks.

Using the proper prompt template (defined by model authors) is necessary to ensure stable and reliable evaluation results. Without it, the evaluation may not reflect accurate results. This only applies to instruction-tuned models, as base models were not fine-tuned to process such templates.

We noticed only a slight variation when modifying precision settings or using 8-bit quantization, suggesting these factors do not strongly account for differing results across papers. However, 4-bit quantization does have a notable effect.

The observed inconsistencies in the HumanEvalFix benchmark scores raise the question of whether other benchmarks are affected by similar issues. While related work has suggested that this is indeed the case [9–12], we further support this claim through an addi-

tional small-scale study. To this end, we reviewed further literature to investigate whether benchmark result inconsistencies also appear more broadly. We specifically focused on MBPP and its improved variant, MBPP+. We limited our investigation to DeepSeekCoder models. The reported scores across papers are summarized in Table 2. The discrepancies in these scores indicate that challenges in benchmark reproducibility extend beyond bugfixing on the HumanEvalFix benchmark.

Table 2. Reported MBPP and MBPP+ results of DeepSeekCoder variants.

Model	MBPP	MBPP+
DeepSeekCoder-Inst (33B) [3]	61 [36], 66 [37], 73.2 [22], 78.7 [38,39], 80.4 [40–42]	59.1 [22], 66.7 [38,39], 70.1 [40–42]
DeepSeekCoder-Inst (6.7B) [3]	60.8 [36], 70.2 [22], 72.7 [38], 73.2 [39], 74.3 [43], 74.9 [40–42]	56.6 [22], 63.4 [38,39], 65.1 [43], 65.6 [40–42]
DeepSeekCoder-Inst (1.3B) [3]	55.4 [22], 63.7 [38], 65.3 [41]	46.9 [22], 53.1 [38], 54.8 [41]
DeepSeekCoder (33B) [3]	74.2 [41,42]	60.7 [41,42]
DeepSeekCoder (6.7B) [3]	70.2 [38,39,41,42]	51.6 [42], 56.6 [38,39,41], 66.4 [44]
DeepSeekCoder (1.3B) [3]	55.4 [38], 55.6 [41]	46.9 [38,41]

To ensure reproducibility and comparability in future research, we propose the following standardized evaluation checklist:

1. Use the same precision as specified in the original model release (*bf16* for most models) or possibly *fp32* to reduce the probability of rounding errors.
2. Avoid model quantization if possible.
3. Follow the prompt formatting recommended by model authors.
4. Unless stated otherwise by the model authors, use greedy decoding for generation stability or generate more samples (such as $n = 20$) to provide a more refined pass@1 estimate.
5. Ensure that limits to generation length are sufficiently large to allow complete responses.
6. Report the exact model name, version, tuning, number of parameters.
7. Provide details about the used evaluation framework, evaluation settings, and benchmark dataset version.

6. Conclusions

In this paper, we highlighted the issue of inconsistent benchmark results in the literature on the HumanEvalFix benchmark, one of the most widely used benchmarks for evaluating bugfixing. We found that for most of the models mentioned in multiple papers, the reported benchmark results can vary significantly. We conducted evaluations to determine the effect of various evaluation settings and to reproduce different scores reported for the same models, in order to uncover the reasons for these inconsistencies.

Through a series of empirical evaluations, using multiple models in different sizes and tunings, we identified the set of factors influencing benchmark results. Our experiments revealed that factors such as the prompt template, maximum generation length, and decoding strategies had a notable influence on the benchmark results, whereas precision and 8-bit quantization did not. We have also found cases where results were likely misattributed between instruction-tuned and base models. We wish to emphasize the importance of using appropriate evaluation settings and including these settings in the studies to ensure reliable and reproducible results.

Broader Impact

We hope that by giving a clearer picture of the capabilities of LLMs, our research will have the positive impact of helping both academic and industrial stakeholders choose more appropriate models. Collectively, these decisions could influence both academic and commercial adoption of LLMs towards a more evidence-based and better direction.

Our environmental impact was relatively small: the experiments consumed roughly 100 GPU-hours on two Nvidia A100 GPUs. We believe that even this impact could be offset by other researchers not having to waste GPU cycles chasing irreproducible results.

Author Contributions: Conceptualization, B.S.; methodology, B.S.; software, B.M.; investigation, B.S. and B.M.; writing—original draft preparation, B.S.; writing—review and editing, B.S., B.M., B.P., and T.G.; visualization, B.M.; supervision, B.P. and T.G.; project administration, T.G.; funding acquisition, T.G. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by EKÖP-24 University Excellence Scholarship Program of the Ministry for Culture and Innovation from the Source of the National Research, Development and Innovation Fund.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The benchmark data used in the study—the HumanEvalFix dataset—are openly available as part of the HumanEvalPack, which is part of OctoPack. The relevant paper is available at https://doi.org/10.48550/arXiv.2308.07124 (accessed on 19 May 2025), while the dataset itself is available at https://github.com/bigcode-project/octopack (accessed on 19 May 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A

Appendix A.1. Prompt Template Formats

Here, we document the five prompt templates used in this study for instructing large language models to fix bugs in a Python function. The code to be fixed is left incomplete for better transparency. The final ... marks where the model should start generating the fixed code.

1. Instruct Prompt (General Instruction Style)

```
from typing import List
def has_close_elements(numbers: List[float], threshold: float) -> bool:
    ...
def check(has_close_elements):
    ...
check(has_close_elements)
Fix bugs in has_close_elements.
...
```

2. DeepSeekCoder Prompt

```
You are an AI programming assistant, utilizing the Deepseek Coder model,
developed by Deepseek Company, and you only answer questions related to
computer science. For politically sensitive questions, security and privacy
issues, and other non-computer science questions, you will refuse to answer
### Instruction:
Fix bugs in has_close_elements.
from typing import List
def has_close_elements(numbers: List[float], threshold: float) -> bool:
```

```
    ...
def check(has_close_elements):
    ...
check(has_close_elements)
### Response:
...
```

3. CodeLlama Prompt

```
[INST] Fix bugs in has_close_elements.
from typing import List
def has_close_elements(numbers: List[float], threshold: float) -> bool:
    ...
def check(has_close_elements):
    ...
check(has_close_elements) [/INST] ...
```

4. CodeGemma Prompt

```
<start_of_turn>user
Fix bugs in has_close_elements.
from typing import List
def has_close_elements(numbers: List[float], threshold: float) -> bool:
    ...
def check(has_close_elements):
    ...
check(has_close_elements)<end_of_turn>
<start_of_turn>model
...
```

5. Wizardcoder Prompt

```
Below is an instruction that describes a task. Write a response that
appropriately completes the request.
### Instruction:
Fix bugs in has_close_elements.
from typing import List
def has_close_elements(numbers: List[float], threshold: float) -> bool:
    ...
def check(has_close_elements):
    ...
check(has_close_elements)
### Response:
...
```

Appendix A.2. Links to GitHub Issues

Here, we provide links to the reviewed issues that were about the unsuccessful reproduction of officially reported results. Reviewing these issues helped us determine some of the potential causes of differences in benchmark results.

- A differently tuned version of the model was used:
  - https://github.com/bigcode-project/bigcode-evaluation-harness/issues/82 (accessed on 5 November 2024)
  - https://github.com/bigcode-project/bigcode-evaluation-harness/issues/228 (accessed on 5 November 2024)
- The wrong benchmark variant was used:
  - https://github.com/bigcode-project/bigcode-evaluation-harness/issues/159 (accessed on 5 November 2024)

  - https://github.com/bigcode-project/bigcode-evaluation-harness/issues/246 (accessed on 5 November 2024)
- The temperature was set incorrectly:
  - https://github.com/bigcode-project/bigcode-evaluation-harness/issues/142 (accessed on 5 November 2024)
- An incorrect prompt was used:
  - https://github.com/bigcode-project/bigcode-evaluation-harness/issues/262 (accessed on 5 November 2024)
- The reproduced results were different by only a few percentage points compared to official reports:
  - https://github.com/bigcode-project/bigcode-evaluation-harness/issues/165 (accessed on 5 November 2024)
  - https://github.com/bigcode-project/bigcode-evaluation-harness/issues/220 (accessed on 5 November 2024)
  - https://github.com/bigcode-project/bigcode-evaluation-harness/issues/233 (accessed on 5 November 2024)

References

1. Muennighoff, N.; Liu, Q.; Zebaze, A.; Zheng, Q.; Hui, B.; Zhuo, T.Y.; Singh, S.; Tang, X.; Werra, L.V.; Longpre, S. OctoPack: Instruction Tuning Code Large Language Models. In Proceedings of the NeurIPS 2023 Workshop on Instruction Tuning and Instruction, New Orleans, LA, USA, 15 December 2023
2. Rozière, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.E.; Adi, Y.; Liu, J.; Sauvestre, R.; Remez, T.; et al. Code Llama: Open Foundation Models for Code. *arXiv* **2024**, arXiv:2308.12950.
3. Guo, D.; Zhu, Q.; Yang, D.; Xie, Z.; Dong, K.; Zhang, W.; Chen, G.; Bi, X.; Wu, Y.; Li, Y.K.; et al. DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence. *arXiv* **2024**, arXiv:2401.14196.
4. Lin, D.; Koppel, J.; Chen, A.; Solar-Lezama, A. QuixBugs: A multi-lingual program repair benchmark set based on the quixey challenge. In Proceedings of the Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity, New York, NY, USA, 22–27 October 2017; pp. 55–56. [CrossRef]
5. Widyasari, R.; Sim, S.Q.; Lok, C.; Qi, H.; Phan, J.; Tay, Q.; Tan, C.; Wee, F.; Tan, J.E.; Yieh, Y.; et al. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, New York, NY, USA, 8–13 November 2020; pp. 1556–1560. [CrossRef]
6. Liu, S.; Chai, L.; Yang, J.; Shi, J.; Zhu, H.; Wang, L.; Jin, K.; Zhang, W.; Zhu, H.; Guo, S.; et al. MdEval: Massively Multilingual Code Debugging. *arXiv* **2024**, arXiv:2411.02310.
7. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; de Oliveira Pinto, H.P.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating Large Language Models Trained on Code. *arXiv* **2021**, arXiv:2107.03374.
8. Wang, S.; Asilis, J.; Ömer, F.A.; Bilgin, E.B.; Liu, O.; Neiswanger, W. Tina: Tiny Reasoning Models via LoRA. *arXiv* **2025**, arXiv:2504.15777.
9. Biderman, S.; Schoelkopf, H.; Sutawika, L.; Gao, L.; Tow, J.; Abbasi, B.; Aji, A.F.; Ammanamanchi, P.S.; Black, S.; Clive, J.; et al. Lessons from the Trenches on Reproducible Evaluation of Language Models. *arXiv* **2024**, arXiv:2405.14782.
10. Laskar, M.T.R.; Alqahtani, S.; Bari, M.S.; Rahman, M.; Khan, M.A.M.; Khan, H.; Jahan, I.; Bhuiyan, A.; Tan, C.W.; Parvez, M.R.; et al. A Systematic Survey and Critical Review on Evaluating Large Language Models: Challenges, Limitations, and Recommendations. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Miami, FL, USA, 12–16 November 2024; pp. 13785–13816. [CrossRef]
11. Hochlehnert, A.; Bhatnagar, H.; Udandarao, V.; Albanie, S.; Prabhu, A.; Bethge, M. A Sober Look at Progress in Language Model Reasoning: Pitfalls and Paths to Reproducibility. *arXiv* **2025**, arXiv:2504.07086.
12. Yuan, J.; Li, H.; Ding, X.; Xie, W.; Li, Y.J.; Zhao, W.; Wan, K.; Shi, J.; Hu, X.; Liu, Z. Give Me FP32 or Give Me Death? Challenges and Solutions for Reproducible Reasoning. *arXiv* **2025**, arXiv:2506.09501.
13. Team, C.; Zhao, H.; Hui, J.; Howland, J.; Nguyen, N.; Zuo, S.; Hu, A.; Choquette-Choo, C.A.; Shen, J.; Kelley, J.; et al. CodeGemma: Open Code Models Based on Gemma. *arXiv* **2024**, arXiv:2406.11409.
14. Luo, Z.; Xu, C.; Zhao, P.; Sun, Q.; Geng, X.; Hu, W.; Tao, C.; Ma, J.; Lin, Q.; Jiang, D. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *arXiv* **2023**, arXiv:2306.08568.

15. Ben Allal, L.; Muennighoff, N.; Kumar Umapathi, L.; Lipkin, B.; von Werra, L. A Framework for the Evaluation of Code Generation Models. 2022. Available online: https://github.com/bigcode-project/bigcode-evaluation-harness (accessed on 5 November 2024).
16. Cassano, F.; Li, L.; Sethi, A.; Shinn, N.; Brennan-Jones, A.; Lozhkov, A.; Anderson, C.J.; Guha, A. Can It Edit? Evaluating the Ability of Large Language Models to Follow Code Editing Instructions. In Proceedings of the Conference on Language Modelling (COLM), Philadelphia, PA, USA, 7–9 October 2024.
17. Chae, H.; Kwon, T.; Moon, S.; Song, Y.; Kang, D.; Ong, K.T.i.; Kwak, B.w.; Bae, S.; Hwang, S.w.; Yeo, J. Coffee-Gym: An Environment for Evaluating and Improving Natural Language Feedback on Erroneous Code. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*; Al-Onaizan, Y., Bansal, M., Chen, Y.N., Eds.; Association for Computational Linguistics: Miami, FL, USA, 2024; pp. 22503–22524.
18. Dehghan, M.; Wu, J.J.; Fard, F.H.; Ouni, A. MergeRepair: An Exploratory Study on Merging Task-Specific Adapters in Code LLMs for Automated Program Repair. *arXiv* **2024**, arXiv:2408.09568.
19. Campos, V. Bug Detection and Localization using Pre-trained Code Language Models. In *INFORMATIK 2024*; Gesellschaft für Informatik e.V.: Bonn, Germany, 2024; pp. 1419–1429. [CrossRef]
20. Jiang, Y.; He, Q.; Zhuang, X.; Wu, Z. Code Comparison Tuning for Code Large Language Models. *arXiv* **2024**, arXiv:2403.19121.
21. Jiang, H.; Liu, Q.; Li, R.; Ye, S.; Wang, S. CursorCore: Assist Programming through Aligning Anything. *arXiv* **2024**, arXiv:2410.07002.
22. Lozhkov, A.; Li, R.; Allal, L.B.; Cassano, F.; Lamy-Poirier, J.; Tazi, N.; Tang, A.; Pykhtar, D.; Liu, J.; Wei, Y.; et al. StarCoder 2 and The Stack v2: The Next Generation. *arXiv* **2024**, arXiv:2402.19173.
23. Mishra, M.; Stallone, M.; Zhang, G.; Shen, Y.; Prasad, A.; Soria, A.M.; Merler, M.; Selvam, P.; Surendran, S.; Singh, S.; et al. Granite Code Models: A Family of Open Foundation Models for Code Intelligence. *arXiv* **2024**, arXiv:2405.04324.
24. Moon, S.; Chae, H.; Song, Y.; Kwon, T.; Kang, D.; iunn Ong, K.T.; won Hwang, S.; Yeo, J. Coffee: Boost Your Code LLMs by Fixing Bugs with Feedback. *arXiv* **2024**, arXiv:2311.07215.
25. Nakamura, T.; Mishra, M.; Tedeschi, S.; Chai, Y.; Stillerman, J.T.; Friedrich, F.; Yadav, P.; Laud, T.; Chien, V.M.; Zhuo, T.Y.; et al. Aurora-M: Open Source Continual Pre-training for Multilingual Language and Code. In Proceedings of the 31st International Conference on Computational Linguistics: Industry Track, Abu Dhabi, United Arab Emirates, 19–24 January 2025; Rambow, O., Wanner, L., Apidianaki, M., Al-Khalifa, H., Eugenio, B.D., Schockaert, S., Darwish, K., Agarwal, A., Eds.; Association for Computational Linguistics: Abu Dhabi, United Arab Emirates, 2025; pp. 656–678.
26. Shi, Y.; Wang, S.; Wan, C.; Gu, X. From Code to Correctness: Closing the Last Mile of Code Generation with Hierarchical Debugging. *arXiv* **2024**, arXiv:2410.01215.
27. Singhal, M.; Aggarwal, T.; Awasthi, A.; Natarajan, N.; Kanade, A. NoFunEval: Funny How Code LMs Falter on Requirements Beyond Functional Correctness. *arXiv* **2024**, arXiv:2401.15963.
28. Wang, X.; Li, B.; Song, Y.; Xu, F.F.; Tang, X.; Zhuge, M.; Pan, J.; Song, Y.; Li, B.; Singh, J.; et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *arXiv* **2024**, arXiv:2407.16741.
29. Yang, J.; Jimenez, C.E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; Press, O. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Proceedings of the Advances in Neural Information Processing Systems*; Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., Zhang, C., Eds.; Curran Associates, Inc.: Red Hook, NY, USA, 2024; Volume 37, pp. 50528–50652.
30. Yu, Z.; Zhang, X.; Shang, N.; Huang, Y.; Xu, C.; Zhao, Y.; Hu, W.; Yin, Q. WaveCoder: Widespread And Versatile Enhancement For Code Large Language Models By Instruction Tuning. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Bangkok, Thailand, 11–16 August 2024*; Ku, L.W., Martins, A., Srikumar, V., Eds.; Association for Computational Linguistics: Bangkok, Thailand, 2024; pp. 5140–5153. [CrossRef]
31. Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Yang, A.; Fan, A.; et al. The Llama 3 Herd of Models. *arXiv* **2024**, arXiv:2407.21783.
32. BigScience Workshop; Scao, T.L.; Fan, A.; Akiki, C.; Pavlick, E.; Ilić, S.; Hesslow, D.; Castagné, R.; Luccioni, A.S.; Yvon, G.; et al. BLOOM: A 176B-Parameter Open-Access Multilingual Language Model. *arXiv* **2023**, arXiv:2211.05100.
33. Li, R.; Allal, L.B.; Zi, Y.; Muennighoff, N.; Kocetkov, D.; Mou, C.; Marone, M.; Akiki, C.; Li, J.; Chim, J.; et al. StarCoder: May the source be with you! *arXiv* **2023**, arXiv:2305.06161.
34. Zheng, Q.; Xia, X.; Zou, X.; Dong, Y.; Wang, S.; Xue, Y.; Shen, L.; Wang, Z.; Wang, A.; Li, Y.; et al. CodeGeeX: A Pre-Trained Model for Code Generation with Multilingual Benchmarking on HumanEval-X. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, New York, NY, USA, 6–10 August 2023; KDD '23; pp. 5673–5684. [CrossRef]
35. Wang, Y.; Le, H.; Gotmare, A.; Bui, N.; Li, J.; Hoi, S. CodeT5+: Open Code Large Language Models for Code Understanding and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*; Bouamor, H., Pino, J., Bali, K., Eds.; Association for Computational Linguistics: Singapore, 2023; pp. 1069–1088. [CrossRef]

36. Wang, Z.Z.; Asai, A.; Yu, X.V.; Xu, F.F.; Xie, Y.; Neubig, G.; Fried, D. CodeRAG-Bench: Can Retrieval Augment Code Generation? In *Proceedings of the Findings of the Association for Computational Linguistics: NAACL 2025*; Chiruzzo, L., Ritter, A., Wang, L., Eds.; Association for Computational Linguistics: Albuquerque, New Mexico, 2025; pp. 3199–3214. [CrossRef]
37. Matton, A.; Sherborne, T.; Aumiller, D.; Tommasone, E.; Alizadeh, M.; He, J.; Ma, R.; Voisin, M.; Gilsenan-McMahon, E.; Gallé, M. On Leakage of Code Generation Evaluation Datasets. In *Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2024*; Al-Onaizan, Y., Bansal, M., Chen, Y.N., Eds.; Association for Computational Linguistics: Miami, FL, USA, 2024; pp. 13215–13223. [CrossRef]
38. Wei, Y.; Wang, Z.; Liu, J.; Ding, Y.; Zhang, L. Magicoder: Empowering code generation with OSS-INSTRUCT. In Proceedings of the 41st International Conference on Machine Learning, JMLR.org, Vienna, Austria, 21–27 July 2024.
39. Zheng, T.; Zhang, G.; Shen, T.; Liu, X.; Lin, B.Y.; Fu, J.; Chen, W.; Yue, X. OpenCodeInterpreter: Integrating Code Generation with Execution and Refinement. In *Proceedings of the Findings of the Association for Computational Linguistics: ACL 2024*; Ku, L.W., Martins, A., Srikumar, V., Eds.; Association for Computational Linguistics: Bangkok, Thailand, 2024; pp. 12834–12859. [CrossRef]
40. Lei, B.; Li, Y.; Chen, Q. AutoCoder: Enhancing Code Large Language Model with AIEV-INSTRUCT. *arXiv* **2024**, arXiv:2405.14906.
41. Hui, B.; Yang, J.; Cui, Z.; Yang, J.; Liu, D.; Zhang, L.; Liu, T.; Zhang, J.; Yu, B.; Lu, K.; et al. Qwen2.5-Coder Technical Report. *arXiv* **2024**, arXiv:2409.12186.
42. Yu, Z.; Zhao, Y.; Cohan, A.; Zhang, X.P. HumanEval Pro and MBPP Pro: Evaluating Large Language Models on Self-invoking Code Generation. *arXiv* **2024**, arXiv:2412.21199.
43. Miao, Y.; Gao, B.; Quan, S.; Lin, J.; Zan, D.; Liu, J.; Yang, J.; Liu, T.; Deng, Z. Aligning CodeLLMs with Direct Preference Optimization. *arXiv* **2024**, arXiv:2410.18585.
44. Dou, S.; Jia, H.; Wu, S.; Zheng, H.; Zhou, W.; Wu, M.; Chai, M.; Fan, J.; Huang, C.; Tao, Y.; et al. What's Wrong with Your Code Generated by Large Language Models? An Extensive Study. *arXiv* **2024**, arXiv:2407.06153.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.